我将提供一个基于 **Spring Boot + PostGIS + Geotools** 的 **PBF 矢量瓦片预切片系统**，支持并行切片、任务监控、进度查询，并保持前后端 **EPSG:4326** 坐标系统一。

---

1. 总体架构

```
[User] → REST API → TaskController
                        ↓
                   TileService (提交/查询任务)
                        ↓
                   TileTaskExecutor (异步执行)
                        ↓
      ┌──────────────────────────────────┐
      │  线程池 (并行)                    │
      │  每瓦片 → 生成MVT → 写文件        │
      └──────────────────────────────────┘
                        ↓
                PostgreSQL (ST_AsMVT)
                        ↓
    Geotools (读取数据源元信息)
```

关键组件

- **TaskEntity**：记录切片任务（zoom范围、bbox、状态、进度）
- **TileService**：计算瓦片总数、启动异步任务、更新进度
- **GeotoolsService**：利用 Geotools 连接 PostGIS 获取表边界/坐标系
- **MVTGenerator**：执行单个瓦片的 SQL 并存储 `.pbf` 文件
- **Controller**：提交任务、查询进度

---

2. 依赖配置 (pom.xml)

```xml
<parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.2.0</version>
</parent>

<properties>
    <geotools.version>30.0</geotools.version>
</properties>

<dependencies>
    <!-- Spring Boot -->
```

```xml
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>

    <!-- PostgreSQL & PostGIS -->
    <dependency>
        <groupId>org.postgresql</groupId>
        <artifactId>postgresql</artifactId>
    </dependency>
    <dependency>
        <groupId>net.postgis</groupId>
        <artifactId>postgis-jdbc</artifactId>
        <version>2021.1.0</version>
    </dependency>

    <!-- Geotools (用于获取数据源元信息) -->
    <dependency>
        <groupId>org.geotools</groupId>
        <artifactId>gt-main</artifactId>
        <version>${geotools.version}</version>
    </dependency>
    <dependency>
        <groupId>org.geotools</groupId>
        <artifactId>gt-jdbc-postgis</artifactId>
        <version>${geotools.version}</version>
    </dependency>
    <dependency>
        <groupId>org.geotools</groupId>
        <artifactId>gt-data</artifactId>
        <version>${geotools.version}</version>
    </dependency>

    <!-- Lombok（可选） -->
    <dependency>
        <groupId>org.projectlombok</groupId>
        <artifactId>lombok</artifactId>
        <optional>true</optional>
    </dependency>
</dependencies>
```

3. 数据库实体与任务表

3.1 任务实体

```java
@Entity
@Table(name = "tile_tasks")
@Data
public class TileTask {
    @Id
    @GeneratedValue(strategy = GenerationType.UUID)
    private String id;

    private String tableName;
    private String geometryColumn = "geom";
    private int srid = 4326;

    private int minZoom;
    private int maxZoom;

    // 经纬度范围（可选，留空则自动探测）
    private Double minLon;
    private Double minLat;
    private Double maxLon;
    private Double maxLat;

    private String outputDir;

    // 状态：PENDING, RUNNING, COMPLETED, FAILED, CANCELLED
    @Enumerated(EnumType.STRING)
    private TaskStatus status = TaskStatus.PENDING;

    private int totalTiles;
    private int completedTiles;
    private int failedTiles;

    private LocalDateTime createdAt = LocalDateTime.now();
    private LocalDateTime updatedAt;

    // 错误信息（最多存前N条）
    @Column(length = 4000)
    private String errorLog;
}
```

3.2 仓库

```java
public interface TileTaskRepository extends JpaRepository<TileTask, String> {
}
```

---

4. 核心服务

4.1 Geotools 元数据服务

```
@Service
public class GeotoolsService {

    @Value("${spring.datasource.url}")
    private String dbUrl;
    @Value("${spring.datasource.username}")
    private String dbUser;
    @Value("${spring.datasource.password}")
    private String dbPassword;

    /**
     * 获取指定表的要素源，用于读取元数据
     */
    public SimpleFeatureSource getFeatureSource(String tableName, String
geometryColumn) throws IOException {
        Map<String, Object> params = new HashMap<>();
        params.put(PostgisNGDataStoreFactory.DBTYPE.key, "postgis");
        params.put(PostgisNGDataStoreFactory.HOST.key, extractHost(dbUrl));
        params.put(PostgisNGDataStoreFactory.PORT.key, extractPort(dbUrl));
        params.put(PostgisNGDataStoreFactory.DATABASE.key, extractDatabase(dbUrl));
        params.put(PostgisNGDataStoreFactory.USER.key, dbUser);
        params.put(PostgisNGDataStoreFactory.PASSWD.key, dbPassword);
        params.put(PostgisNGDataStoreFactory.SCHEMA.key, "public");

        DataStore dataStore = DataStoreFinder.getDataStore(params);
        if (dataStore == null) throw new IOException("无法连接 PostGIS 数据库");
        String typeName = tableName;
        SimpleFeatureSource featureSource = dataStore.getFeatureSource(typeName);
        if (featureSource == null) throw new IOException("未找到表: " + tableName);
        return featureSource;
    }

    /**
     * 获取表的经纬度边界（自动转换为4326）
     */
    public double[] getBoundsIn4326(SimpleFeatureSource source) throws IOException {
        ReferencedEnvelope env = source.getBounds();
        CoordinateReferenceSystem sourceCrs =
source.getSchema().getCoordinateReferenceSystem();
        CoordinateReferenceSystem targetCrs = CRS.decode("EPSG:4326");
        if (!CRS.equalsIgnoreMetadata(sourceCrs, targetCrs)) {
            env = env.transform(targetCrs, true);
        }
        return new double[]{env.getMinX(), env.getMinY(), env.getMaxX(),
env.getMaxY()};
    }

    // 辅助方法：从 jdbc url 提取 host/port/database
}
```

4.2 瓦片范围计算与总数

```
@Component
public class TileGridCalculator {

    /**
     * 根据 bbox (lon/lat) 和 zoom 计算 xy 范围
     */
    public TileRange computeTileRange(double minLon, double minLat, double maxLon,
double maxLat, int zoom) {
        double tileDeg = 180.0 / (1 << zoom);
        int xMin = (int) Math.floor((minLon + 180) / tileDeg);
        int xMax = (int) Math.floor((maxLon + 180) / tileDeg);
        int yMin = (int) Math.floor((90 - maxLat) / tileDeg);
        int yMax = (int) Math.floor((90 - minLat) / tileDeg);

        // 限制在合法范围
        int maxXY = (1 << zoom) - 1;
        int maxX = (1 << (zoom + 1)) - 1;
        xMin = Math.max(0, xMin);
        xMax = Math.min(maxX, xMax);
        yMin = Math.max(0, yMin);
        yMax = Math.min(maxXY, yMax);
        return new TileRange(xMin, xMax, yMin, yMax);
    }

    public int totalTiles(double minLon, double minLat, double maxLon, double maxLat,
int minZoom, int maxZoom) {
        int total = 0;
        for (int z = minZoom; z <= maxZoom; z++) {
            TileRange r = computeTileRange(minLon, minLat, maxLon, maxLat, z);
            total += (r.xMax - r.xMin + 1) * (r.yMax - r.yMin + 1);
        }
        return total;
    }

    @Data
    @AllArgsConstructor
    public static class TileRange { int xMin, xMax, yMin, yMax; }
}
```

4.3 瓦片生成器（单个瓦片 SQL + 文件写入）

```
@Component
@RequiredArgsConstructor
public class MVTGenerator {
    private final JdbcTemplate jdbcTemplate;

    /**
     * 生成一个瓦片并写入文件
```

```
     */
    public void generateTile(String tableName, String geomColumn, int srid, int z, int
x, int y, Path outputPath) {
        String sql = """
            WITH tile_bounds AS (
                SELECT ST_MakeEnvelope(
                    -180 + :x * (180.0 / POWER(2, :z)),
                    90 - (:y + 1) * (180.0 / POWER(2, :z)),
                    -180 + (:x + 1) * (180.0 / POWER(2, :z)),
                    90 - :y * (180.0 / POWER(2, :z)),
                    4326
                ) AS env
            ),
            clipped AS (
                SELECT
                    t.*,
                    ST_AsMVTGeom(
                        ST_Transform(t.%s, 4326),   -- 确保4326
                        tb.env, 4096, 0, true
                    ) AS mvt_geom
                FROM %s t, tile_bounds tb
                WHERE ST_Transform(t.%s, 4326) && tb.env
            )
            SELECT ST_AsMVT(clipped.*, 'default', 4096, 'mvt_geom') AS mvt
            FROM clipped
            """.formatted(geomColumn, tableName, geomColumn);

        List<byte[]> result = jdbcTemplate.query(sql,
                Map.of("x", x, "y", y, "z", z),
                (rs, rowNum) -> rs.getBytes("mvt"));

        if (result.isEmpty()) {
            // 没有数据，可写空瓦片或跳过
            return;
        }
        byte[] mvtBytes = result.get(0);
        try {
            Files.createDirectories(outputPath.getParent());
            Files.write(outputPath, mvtBytes);
        } catch (IOException e) {
            throw new UncheckedIOException("写入瓦片失败: " + outputPath, e);
        }
    }
}
```

4.4 并行任务执行服务

```
@Service
@RequiredArgsConstructor
public class TileTaskExecutor {
    private final TileTaskRepository taskRepo;
```

```
    private final TileGridCalculator calculator;
    private final MVTGenerator generator;
    private final ExecutorService executor;

    // 可通过配置文件控制并发度
    @PostConstruct
    public void init() {
        this.executor = Executors.newFixedThreadPool(
            Runtime.getRuntime().availableProcessors() * 2
        );
    }

    @Async
    public void executeTask(String taskId) {
        TileTask task = taskRepo.findById(taskId).orElseThrow();
        task.setStatus(TaskStatus.RUNNING);
        task.setUpdatedAt(LocalDateTime.now());
        taskRepo.save(task);

        try {
            for (int z = task.getMinZoom(); z <= task.getMaxZoom(); z++) {
                TileGridCalculator.TileRange range = calculator.computeTileRange(
                        task.getMinLon(), task.getMinLat(),
                        task.getMaxLon(), task.getMaxLat(), z);
                for (int x = range.getXMin(); x <= range.getXMax(); x++) {
                    for (int y = range.getYMin(); y <= range.getYMax(); y++) {
                        final int Z = z, X = x, Y = y;
                        executor.submit(() -> processSingleTile(task, Z, X, Y));
                    }
                }
            }
            // 所有任务已提交，但尚未完成 —— 实际需要等待全部完成或由计数判断。
            // 简化：这里不等待直接返回，通过 completed+failed == total 判断完成。
            // 可在每个子任务中检查进度并更新状态。
        } catch (Exception e) {
            task.setStatus(TaskStatus.FAILED);
            taskRepo.save(task);
        }
    }

    private void processSingleTile(TileTask task, int z, int x, int y) {
        try {
            Path tilePath = Path.of(task.getOutputDir(), String.valueOf(z),
String.valueOf(x), y + ".pbf");
            generator.generateTile(task.getTableName(), task.getGeometryColumn(),
                    task.getSrid(), z, x, y, tilePath);
            incrementCompleted(task);
        } catch (Exception e) {
            incrementFailed(task, z, x, y, e.getMessage());
        }
    }

    @Transactional
```

```
    void incrementCompleted(TileTask task) {
        task = taskRepo.findById(task.getId()).orElse(null);
        if (task == null) return;
        task.setCompletedTiles(task.getCompletedTiles() + 1);
        checkAndFinish(task);
        taskRepo.save(task);
    }

    @Transactional
    void incrementFailed(TileTask task, int z, int x, int y, String error) {
        task = taskRepo.findById(task.getId()).orElse(null);
        if (task == null) return;
        task.setFailedTiles(task.getFailedTiles() + 1);
        // 追加错误日志（限制长度）
        String logEntry = String.format("z=%d,x=%d,y=%d: %s\n", z, x, y, error);
        String currentLog = task.getErrorLog() == null ? "" : task.getErrorLog();
        if (currentLog.length() < 3500) {
            task.setErrorLog(currentLog + logEntry);
        }
        checkAndFinish(task);
        taskRepo.save(task);
    }

    private void checkAndFinish(TileTask task) {
        if (task.getCompletedTiles() + task.getFailedTiles() >= task.getTotalTiles()) {
            task.setStatus(task.getFailedTiles() == 0 ? TaskStatus.COMPLETED :
TaskStatus.COMPLETED_WITH_ERRORS);
            task.setUpdatedAt(LocalDateTime.now());
        }
    }
}
```

5. 任务提交与进度查询接口

```
@RestController
@RequestMapping("/api/tasks")
@RequiredArgsConstructor
public class TileTaskController {
    private final TileTaskRepository taskRepo;
    private final TileTaskExecutor executor;
    private final GeotoolsService geotoolsService;
    private final TileGridCalculator calculator;

    @PostMapping
    public ResponseEntity<?> createTask(@RequestBody TaskRequest req) {
        // 1. 验证表并自动获取边界（如未提供）
        double[] bbox;
        try {
```

```
            SimpleFeatureSource source =
geotoolsService.getFeatureSource(req.getTableName(),
                    req.getGeometryColumn() != null ? req.getGeometryColumn() :
"geom");
            bbox = geotoolsService.getBoundsIn4326(source);
        } catch (Exception e) {
            return ResponseEntity.badRequest().body("无法访问表: " + e.getMessage());
        }

        double minLon = req.getMinLon() != null ? req.getMinLon() : bbox[0];
        double minLat = req.getMinLat() != null ? req.getMinLat() : bbox[1];
        double maxLon = req.getMaxLon() != null ? req.getMaxLon() : bbox[2];
        double maxLat = req.getMaxLat() != null ? req.getMaxLat() : bbox[3];

        int total = calculator.totalTiles(minLon, minLat, maxLon, maxLat,
                req.getMinZoom(), req.getMaxZoom());

        TileTask task = new TileTask();
        task.setTableName(req.getTableName());
        task.setGeometryColumn(req.getGeometryColumn() != null ?
req.getGeometryColumn() : "geom");
        task.setSrid(4326); // 我们强制转换为4326
        task.setMinZoom(req.getMinZoom());
        task.setMaxZoom(req.getMaxZoom());
        task.setMinLon(minLon);
        task.setMinLat(minLat);
        task.setMaxLon(maxLon);
        task.setMaxLat(maxLat);
        task.setOutputDir(req.getOutputDir());
        task.setTotalTiles(total);
        task.setStatus(TaskStatus.PENDING);
        task = taskRepo.save(task);

        // 异步执行
        executor.executeTask(task.getId());

        return ResponseEntity.ok(Map.of("taskId", task.getId(), "totalTiles", total));
    }

    @GetMapping("/{taskId}")
    public ResponseEntity<?> getTaskProgress(@PathVariable String taskId) {
        return taskRepo.findById(taskId)
                .map(ResponseEntity::ok)
                .orElse(ResponseEntity.notFound().build());
    }
}
```

请求示例 (POST /api/tasks)

```
{
  "tableName": "places",
```

```
  "minZoom": 0,
  "maxZoom": 5,
  "outputDir": "/data/tiles/places",
  "minLon": -180,   // 可选
  "maxLon": 180,
  "minLat": -90,
  "maxLat": 90
}
```

6. 前端调用预切片结果

预切片完成后，瓦片存储在 `outputDir` 中。可以配置一个简单的静态资源映射：

```
@Configuration
public class WebConfig implements WebMvcConfigurer {
    @Value("${tiles.base-dir}") // 如 /data/tiles
    private String tilesBaseDir;

    @Override
    public void addResourceHandlers(ResourceHandlerRegistry registry) {
        registry.addResourceHandler("/tiles/**")
                .addResourceLocations("file:" + tilesBaseDir + "/");
    }
}
```

前端 OpenLayers 代码即可使用 `http://localhost:8080/tiles/{z}/{x}/{y}.pbf` 加载。

7. 关键解释

组件	作用
Geotools	读取 PostGIS 表的 `FeatureSource`，自动获取数据范围（边界框），并支持坐标转换至 EPSG:4326，为未提供 bbox 的任务自动确定切片范围
自定义瓦片网格	基于 `z` 级计算正方形经纬度瓦片（经度跨度 = 纬度跨度），保证前端 VectorTile 渲染无畸变
并行生成	使用线程池并行提交每个瓦片任务，通过数据库计数实现进度监控和完成判断
进度查询	通过 `GET /api/tasks/{taskId}` 返回 JSON，包含 `totalTiles`、`completedTiles`、`failedTiles`、`status` 等

8. 部署与优化建议

- **数据库连接池**：调大 HikariCP 最大连接数（如 `spring.datasource.hikari.maximum-pool-size=20`），避免高并发时等待。
- **文件存储**：建议将瓦片放置于高性能共享存储（如 NFS/OSS），便于 CDN 加速。
- **任务持久化**：任务表记录所有状态，服务重启后可查询历史任务，但不自动恢复（可增加启动后检查 RUNNING 任务并重新提交）。
- **取消任务**：增加 `PUT /api/tasks/{taskId}/cancel` 接口，设置状态为 CANCELLED，并在循环中检查标志。

通过以上设计，即可得到一个 **支持预切片、并行生成、通过 Geotools 自动获取边界、进度监控** 的完整系统，前端配合 OpenLayers EPSG:4326 矢量切片加载，完美实现项目需求。